International University of Sarajevo

Faculty of Engineering and Natural Sciences

Computer Architecture

CS404 SPRING 2025

REPORT

Multicore computer simulation

Students: Hena Šehović

Berina Juković

Berin Žunić

Sarajevo, 24.05.2025.

Contents

1. Introduction

As modern computing demands increasingly faster and more efficient processing, multicore processors have become mainstream. Multicore processors possess two or more separate cores that can run instructions simultaneously, significantly improving performance, multitasking, and power usage. Simulation and modeling are necessary to investigate the internal workings of such architectures, especially in research and educational environments.

One of the key distinctions between recent and traditional processor designs is the distinction between single-core and multicore architectures. (see Figure 1) A single-core processor is able to execute one instruction stream at any given time and thus is limited in its capacity to perform several tasks efficiently. A multicore processor, however, is capable of executing multiple instruction streams concurrently and thus offers better responsiveness and throughput. Parallelism in this case is especially useful for programs like multimedia processing, game-playing, and huge computation.

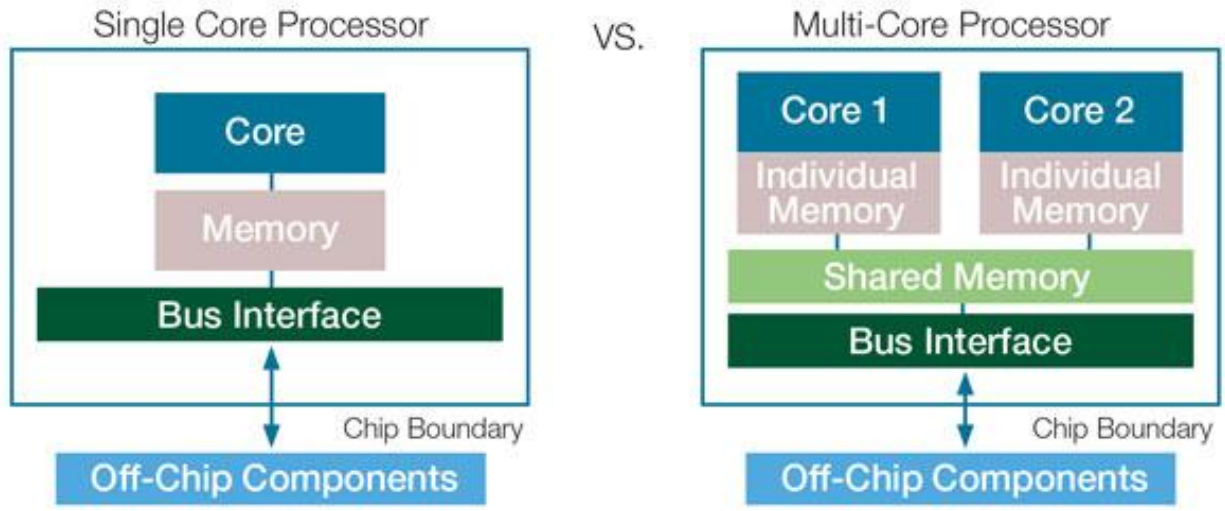

Figure 1: Single Core Processor vs Multicore processor architecture

This project will simulate and design a multicore computer system based on Logisim (Figure 2), a digital logic circuit simulation tool used for teaching. Logisim can be utilized to build digital systems from the bottom up beginning with simple logic gates and components, and thus it provides a suitable platform for simulating the fundamentals of multicore processors.

Figure 2: Logisim logo

The simulation is aimed to demonstrate how multiple processing units can execute in parallel while they use shared memory and communication buses. Instruction execution, memory access, and core-to-core communication are some of the key features that are achieved and experimented upon. The multicore simulation design methodology, architecture, implementation, and output are presented in this report along with some observations regarding multicore computing systems' pitfalls and advantages.

2. Process of making a simulation of Multicore computer in Logisim

2.1 Logical Unit Design

The Logical Unit is a fundamental component of the Arithmetic Logic Unit (ALU), responsible for performing basic bitwise operations. In this project, the logical unit was designed in Logisim to handle three primary operations: AND, OR, and XOR on two 8-bit inputs.

Components used:

1. Two 8-bit input pins: a and b
2. One 2-bit control pin: f (Function selector)
3. One 8-bit output pin: y (Result)
4. 8-bit AND gate
5. 8-bit OR gate
6. 8-bit XOR gate
7. 4-to-1 multiplexer

To construct the logical unit, two 8-bit input pins labeled “a” and “b” were used to serve as the operands for the logical operations. The outputs of these inputs were connected in parallel to three separate 8-bit logic gates: an AND gate to perform the bitwise AND operation, an OR gate for the bitwise OR, and an XOR gate for the bitwise XOR operation. The outputs of these three gates were then connected to a 4-to-1 multiplexer, which serves as a selector to choose which of the logic operation results should be passed on to the final output. The selection is controlled by a 2-bit input pin labeled “f”, which determines the active operation: 00 selects AND, 01 selects OR, 10 selects XOR, and 11 is reserved for potential future expansion. The final selected result from the multiplexer is then directed to an 8-bit output pin labeled “y”, representing the output of the logical unit.

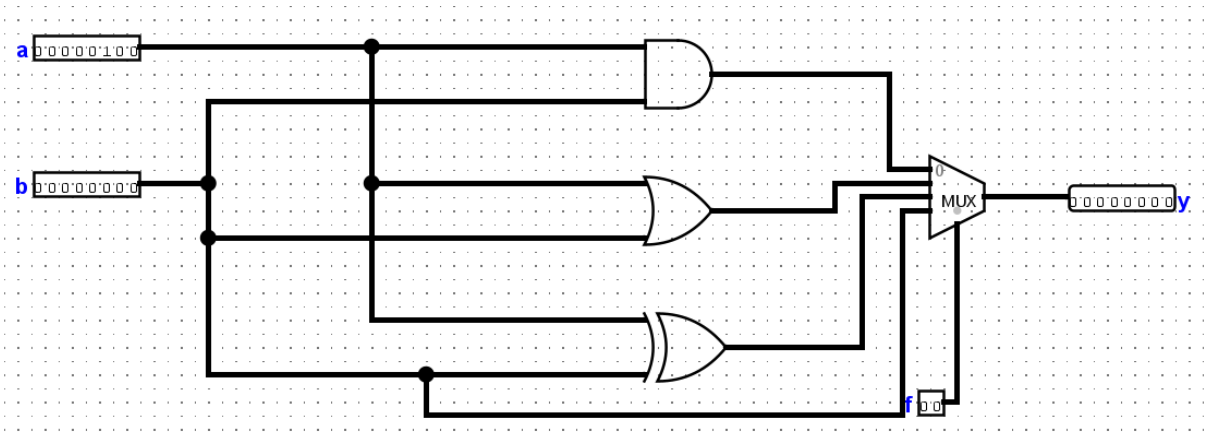

Figure 3: Fully assembled logical unit

The logical unit effectively performs the selected bitwise operation on two 8-bit operands and outputs the result in real-time. This modular design also allows easy integration with the arithmetic unit to form a complete ALU.

2.2 Arithmetic Unit Design

The Arithmetic Unit is a fundamental part of the Arithmetic Logic Unit (ALU) that carries out mathematical calculations. In the simplest implementations, it does binary addition, subtraction, and so on with digital logic circuits. For the project, the arithmetic unit is specialized for 8-bit binary addition with provisions for carry-in, carry-out, and overflow detection.

Components used:

1. Two 8-bit input pins: a and b
2. Two Splitters to split 8-bit inputs into 1-bit lines
3. Eight full adders
4. 1-bit Input Pin: cin
5. Splitter: to combine sum outputs from adders into 8-bit result
6. 8-bit Output Pin: sum
7. XOR Gate
8. 1-bit Output Pin: V
9. 1-bit Output Pin: C

The arithmetic unit was designed to perform 8-bit binary addition, serving as the mathematical half of the ALU. It takes two 8-bit operands, “a” and “b”, each provided through separate 8-bit input pins. These inputs were first passed through splitters, which break each 8-bit input into individual 1-bit lines. Each corresponding bit of A and B was then connected to a series of eight 1-bit full adders, enabling bit-wise addition with carry propagation.

A single-bit input pin labeled “cin” was connected to the carry-in of the first (least significant) adder, allowing for addition with an initial carry if needed. The carry-out of each adder was chained to the carry-in of the next, forming a ripple-carry adder structure. The sum outputs from all eight adders were combined using another splitter and connected to an 8-bit output pin labeled “sum”, which provides the final addition result.

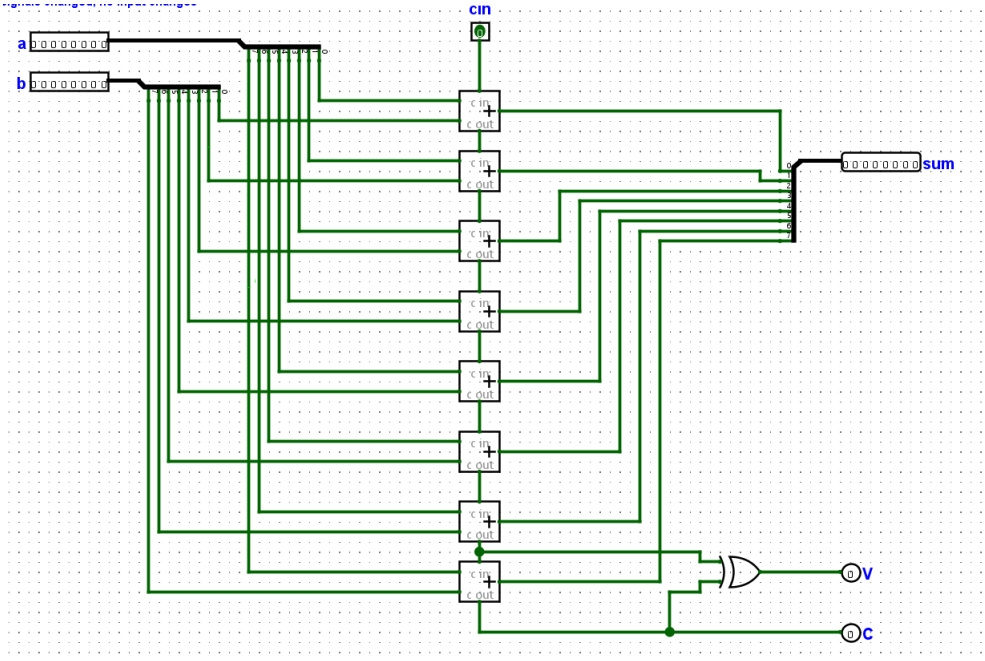

Figure 4: Fully assembled arithmetic unit

To detect arithmetic overflow, the carry-out of the second-to-last and last adder were connected to an XOR gate, and its output was sent to a 1-bit pin labeled V (overflow flag). Additionally, the final carry-out of the last adder was connected to a separate 1-bit output pin labeled C, representing the carry-out of the entire addition.

This arithmetic unit provides a reliable and efficient method for performing 8-bit binary addition within the simulated processor. Its modular design allows for easy integration into the full ALU alongside the logical unit. The inclusion of carry and overflow detection ensures accurate computation, especially in multi-bit operations, making it a crucial component in the execution of arithmetic instructions.

2.3 Arithmetic and Logic Unit (ALU) Design

The Arithmetic Logic Unit (ALU) is the central component responsible for performing both arithmetic and logical operations in a processor. In this project, the ALU was constructed by integrating the previously designed arithmetic and logical units, with additional logic to control operation selection and result handling. The ALU accepts two 8-bit operands and supports configurable operations based on a 4-bit opcode input.

Components used:

1. Two 8-bit input pins: a and b
2. 4-bit input pin: opcode
3. Splitter: to split the 4-bit opcode into individual control bits
4. Two NOT Gates: to generate negated values of a and b
5. Two 2-to-1 Multiplexers: to select between original and negated a and b
6. Arithmetic Unit Component
7. Logical Unit Component
8. Splitter: to extract bits 0 and 2 from the opcode and connect to logical unit's function selector F
9. 2-to-1 Multiplexer: to choose between arithmetic and logical results
10. 8-bit Output Pin: Y
11. Splitter: to break the 8-bit ALU result for status flag generation
12. NOR Gate (8-input)
13. Four 1-bit Output Pins: Z, N, V, C

To begin, two 8-bit input pins labeled "a" and "b" were introduced. Each input was passed through a multiplexer to allow for conditional selection between the original and the negated value of the operand. For operand "a", the original and negated values were connected to a multiplexer controlled by bit 0 of a splitter connected to the 4-bit opcode. Similarly, for operand B, a second multiplexer was used, with its control signal coming from bit 1 of the same splitter. This setup enabled the ALU to support operations like subtraction (via two's complement negation).

The outputs of these two multiplexers were then connected to the inputs of the arithmetic unit. Specifically, the selected "a" value went to the first input, the selected "b" value to the second input, and bit 2 of the opcode splitter was connected to the carry-in (Cin) input of the arithmetic unit.

Simultaneously, the same inputs were also connected to the logical unit. The “a” operand (non-negated) was directly passed to the logical unit, while the selected “b” value from the second multiplexer was used as the second input. The control input “f” of the logical unit was generated by combining bit 0 and bit 2 of the opcode through a new splitter, allowing for selection of logical operations such as AND, OR, and XOR.

The outputs from the arithmetic unit (Sum) and logical unit (Y) were then connected to a third multiplexer, which chooses between the arithmetic or logical result based on bit 3 of the opcode. The output of this multiplexer was sent to an 8-bit output pin labeled “Y”, representing the final result of the ALU operation.

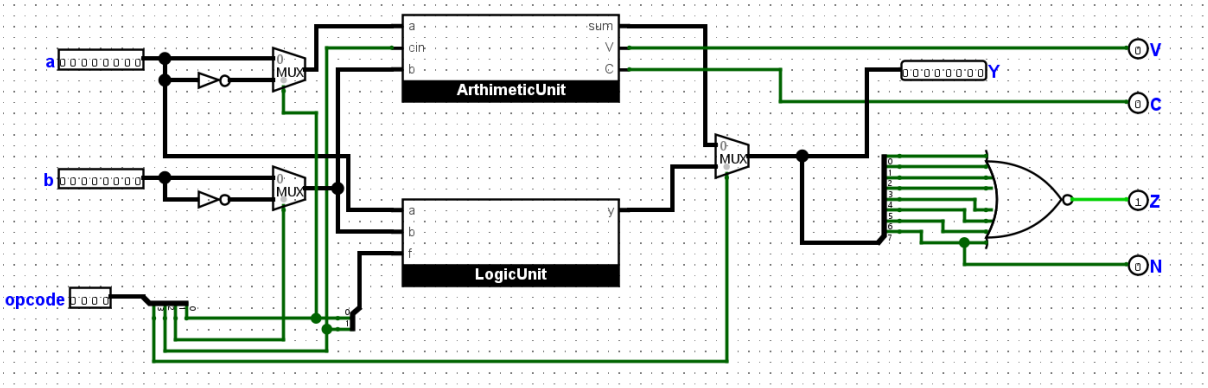

Figure 5: Fully assembled ALU

To generate status flags, the output of the result multiplexer was connected to a splitter. The combined result was passed through a NOR gate to determine if the result is zero (Z flag), which was then connected to a 1-bit output pin Z. Additionally, the most significant bit (bit 7) of the result was extracted and connected to a 1-bit output pin N, representing the negative flag. The overflow (V) and carry-out (C) signals were taken directly from the arithmetic unit and connected to their respective 1-bit output pins.

The ALU serves as a central processing component capable of performing both logical and arithmetic operations based on control signals provided through the opcode. By integrating multiplexers, custom-designed arithmetic and logical units, and additional logic for flag generation, the ALU was made both flexible and extensible. This modular design ensures accurate and efficient processing, making it a critical part of the single-core and multicore systems developed in this simulation.

2.4 Registers Design

The register file is a fundamental component used to temporarily store and access data during processor operations. In this simulation, a register file consisting of eight 8-bit general-purpose registers (R0 to R7) was implemented to support basic read and write operations.

Components used:

1. Eight 8-bit Registers: R0 to R7
2. 8-bit Input Pin: inValue
3. 1-bit Input Pin: write, clk
4. 3-bit Input Pin: Rd, Ra, Rb
5. Demultiplexer (or Inverted Multiplexer)
6. Two 8-to-1 Multiplexers
7. Two 8-bit Output Pins: outA, outB

An 8-bit input pin labeled “inValue” was used to supply data that can be written into any of the registers. The write enable (WE) signals of all registers were controlled using a 3-bit input pin Rd, which specifies the destination register. The Rd input was connected to a demultiplexer (or inverted multiplexer), which activated the correct WE line based on the register index. A 1-bit write pin was used as an enable signal for the demultiplexer, allowing selective register writes. Additionally, a 1-bit clock pin clk was connected to the read clock input (R) of each register to synchronize writes.

For reading, the outputs of all eight registers were connected to two 8-to-1 multiplexers. The first multiplexer was controlled by a 3-bit input pin Ra, and its output was sent to an 8-bit output pin labeled outA, representing the first operand read. The second multiplexer was controlled by a 3-bit input pin Rb, with its output connected to another 8-bit output pin labeled outB, representing the second operand read.

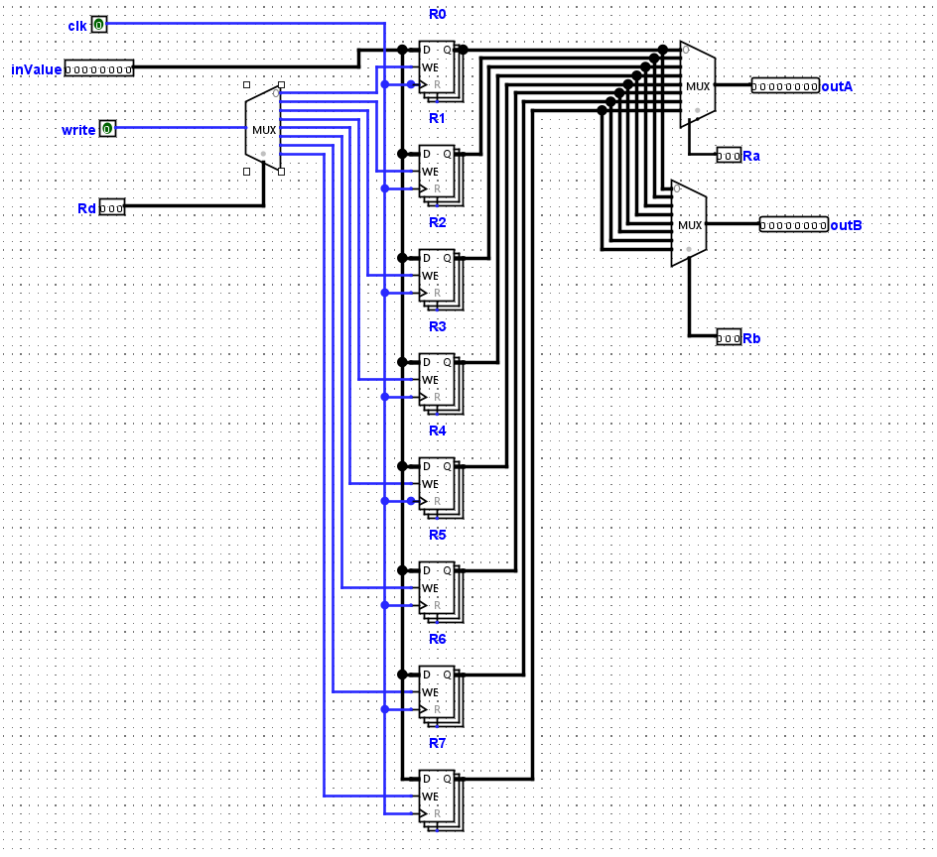

Figure 6: Fully assembled Registers

This setup allows the processor to read two registers simultaneously while writing to a third in the same clock cycle, enabling efficient execution of typical instructions such as arithmetic operations.

The register file provides fast and efficient access to operands needed for computation, supporting simultaneous dual-read and single-write functionality. Its integration with control signals for read and write operations ensures flexibility in instruction execution. This component forms a key building block of the processor, enabling smooth data flow between the ALU and memory or external inputs.

2.5 Core Design

The following figure shows the instruction-fetch and decode stage of a core. The components used in this part are:

1. Clock
2. Logisim-Evolution 10-bit Counter
3. 16-bit ROM
4. 16-bit Register
5. Seven splitters, one is 16-bit, two are 8-bit, one 4-bit, and three 3-bit
6. Four AND gates, with three inputs, each have one or two inputs negated
7. Tunnels for output used later in the simulation

There is a system clock in upper-left corner, it is connected through tunnels to both program counter and instruction register. It is the timing reference that has a role of synchronizing all of core's components. As mentioned it feeds directly to program counter and instruction register, making sure that updates happen at consistent intervals.

Next, the program counter stores the addresses of instructions needed to be executed next. Triggered by clock pulse, it outputs an address that is being sent to instruction memory. The counter is incremented each clock cycle. The output of the program counter is connected to the address input of the instruction memory, whose role is to store the instructions in ROM memory block. It receives the address from program counter, as previously said, and gives the corresponding instruction as output. From instruction memory, the instruction is sent to instruction register.

Instruction register, reads the instruction that is sent from instruction memory at the rising edge of the clock. It is some sort of a buffer between the fetche and decode stages. The instruction is then passed to the control unit. In the control unit the instruction is decoded. And it outputs various signals that are sent to other parts of core, VALUE, ADDER, register selectors (Ra, Rb, Rd), and control flags like , data, load, store, and op which is operation code. The mentioned flags are used for memory and register operations. These green lines in the figure are wires connecting all the components represent the control signal buses, which activate specific components for the current instruction.

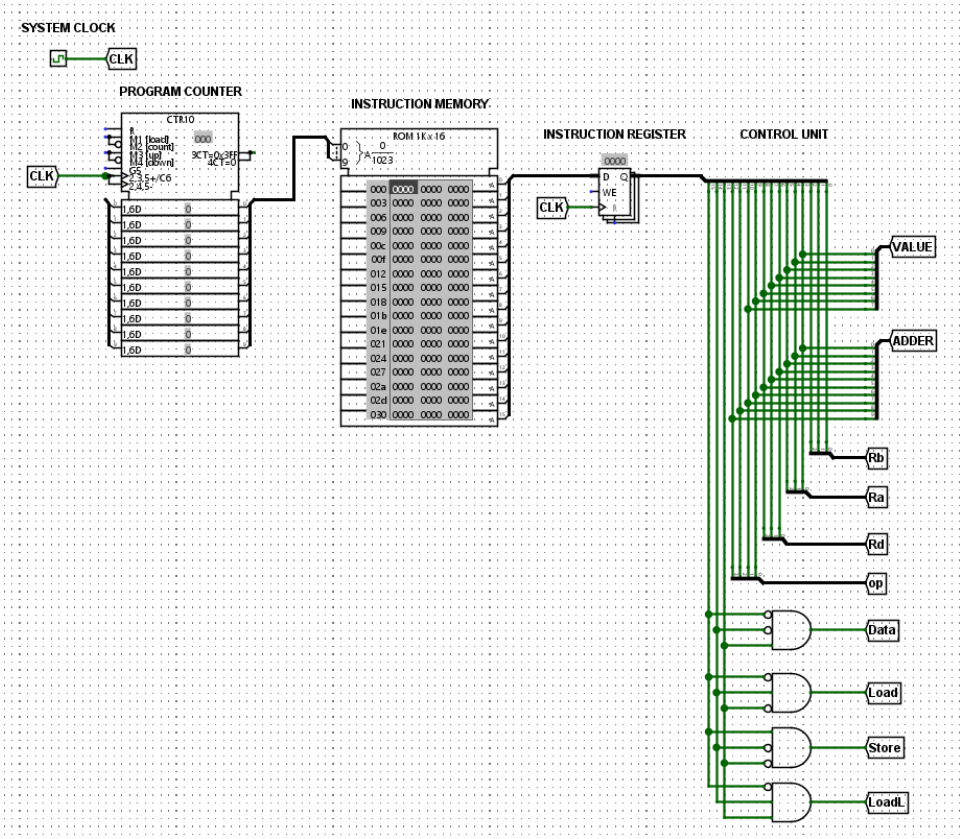

Figure 7: Instruction-fetch and decode stage

In essence, the system starts by clock triggering the program counter updates. Program counter then outputs an address to instruction memory, from that address it then fetches the instruction, which instruction register temporarily holds. Control unit decodes the instruction and produces control signals, which then trigger other components, which were explained in earlier parts.

In the next part of this one-core simulation is the Data Memory. We have used the following components to make this work:

1. 8-bit RAM
2. Two OR gates, one takes two inputs and the other one takes three
3. Register Circuit explained in section 2.4
4. Arthimetic Logic Unit Circuit explained in section 2.3
5. 2-bit register
6. Six controlled buffers
7. Tunnels to make the design more readable

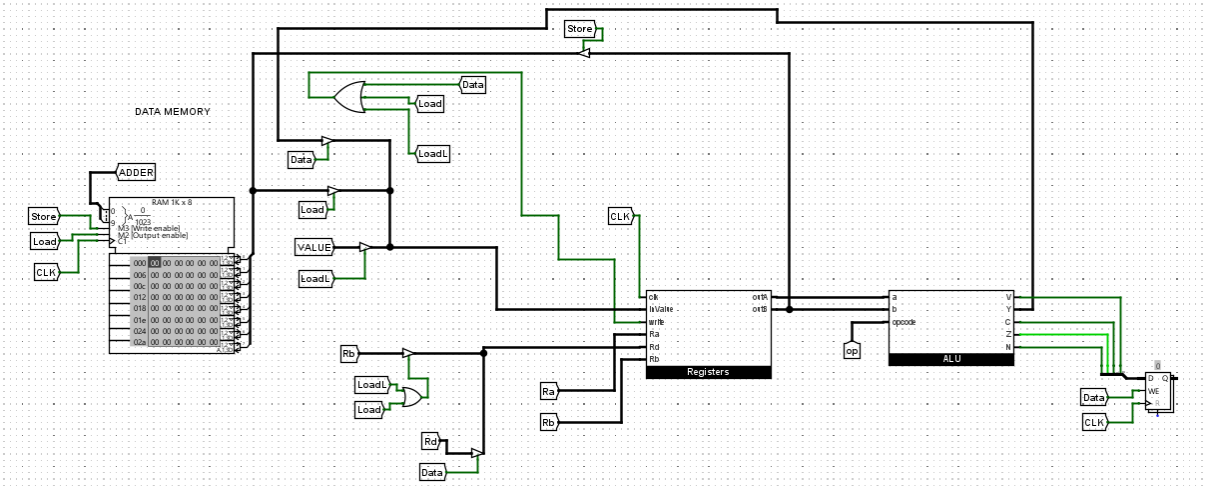

Figure 8: Core Datapath

The figure 8 illustrates the execution and memory access stages of the core. Here, control signals from the Control Unit have been sent from the Register to the ALU for processing. The ALU executes arithmetic or logical operation based on the opcode and outputs the result along. Depending on the instruction, the result of the ALU either returns to a register or is stored in Data Memory. The RAM block communicates with the ALU and other logic using control lines like Load and Store, and its address is typically computed based on either direct output from the ALU or an address from the Adder tunnel. This arrangement can support flexible instruction execution sequences with sufficient synchronization through the system clock.

2.6 Multicore Processor Design

After the single-core CPU design was stable and working, the task was to extend the architecture to emulate a multicore processor. This was done by replicating several instances of identical CPU cores and connecting them to a shared memory system. The idea was to demonstrate how numerous cores could be made to run in parallel, executing separate streams of instructions while synchronizing access to shared resources.

To simulate from single-core to multicore platform, two processor cores were designed with the same architecture mentioned above. They are independent but they share high-level components like instruction memory and data memory. Each of them has its own control unit, ALU, register file, and internal logic so that they can execute instructions independently and in parallel.

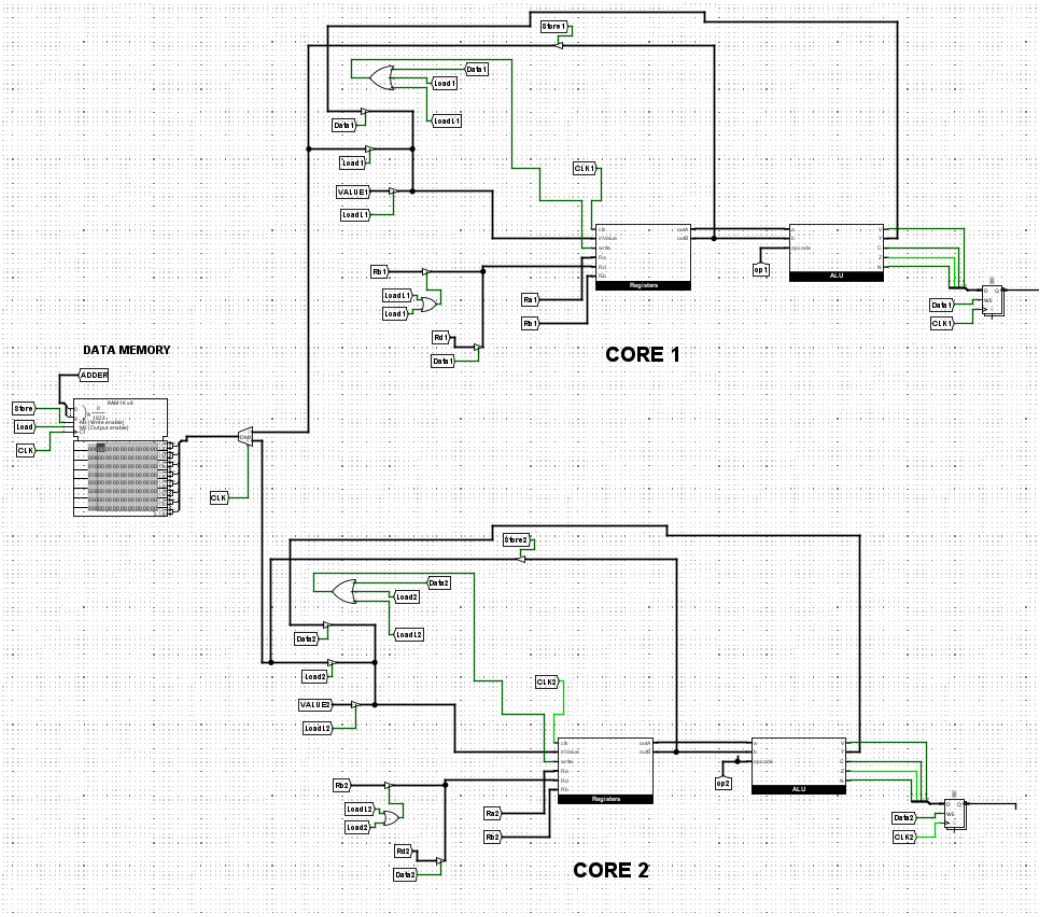

Figure 9: CPU with Two Cores

In order to accommodate more than one core, the instruction memory module had to be revised. Rather than one output to supply instructions to one CPU, the ROM module was designed with multiple outputs, one to each core. This enables each core's control unit to read and decode instructions separately. While the cores are accessing the same ROM, they each have their own program counter, so they can be executing different sections of the same or different programs simultaneously.

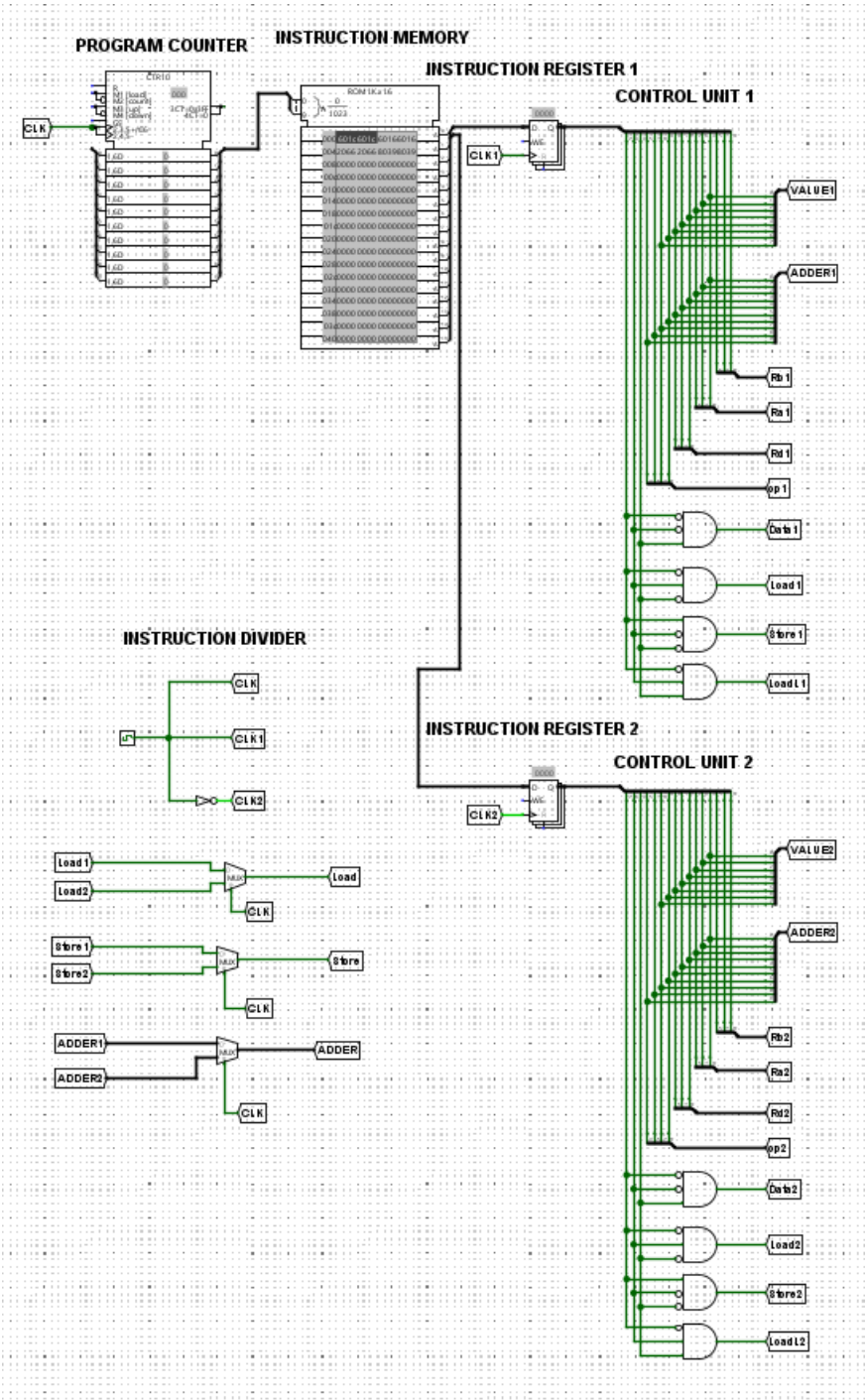

Figure 10: Instruction Memory

Synchronizing access of the shared data memory is the most important issue in the multicore system. Data corruption or undefined behavior is exhibited when both cores are trying to read and write to memory at the same time without synchronization. A multiplexer and demultiplexer-based simple arbitration mechanism was designed to address this.

A multiplexer is utilized to determine which core's output is written to shared memory in a clock cycle.

A demultiplexer will make sure the memory output is sent to the correct core depending on which one triggered the read operation. To prevent contention due to simultaneous access, a clock-based time-sharing scheme was employed. Under this scheme, alternating clock cycles yield to memory access for each core in sequence. Although adequate for a dual-core design, this strategy is limited in its scalability and efficiency. Additional cores would require a more sophisticated memory controller with priority queues or bus locking.

Due to the synchronous clocking and split control paths, instructions can be run in parallel in both cores with minimum interference only at memory access. Tunnels and control signal buses guarantee that each instruction's path—from fetch to execution and memory access—is isolated per core, except for accessing the shared memory.

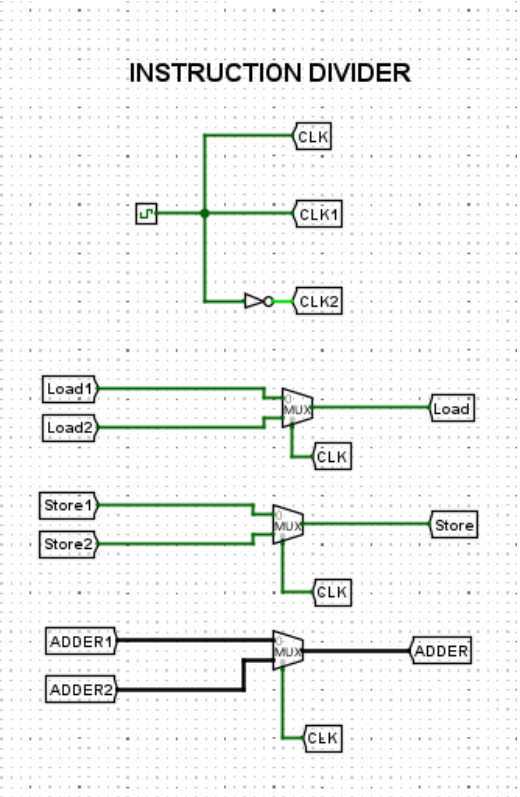

Figure 11: Instruction Divider

This two-core simulation nicely illustrates the basic concepts of multicore communication, synchronization, and shared resource control. It can be used as a building block for more sophisticated extensions of this work, in which more complex scheduling, pipelining, and inter-core communication mechanisms can be investigated.

3. Code-based simulation in Python

To supplement the hardware-level simulation constructed in Logisim, an interactive software simulation of a multicore processor was constructed in Python using the Pygame library. The simulation offers a graphical and dynamic illustration of how multiple cores run concurrently, pass data from shared memory, and access system resources such as caches and a shared bus.

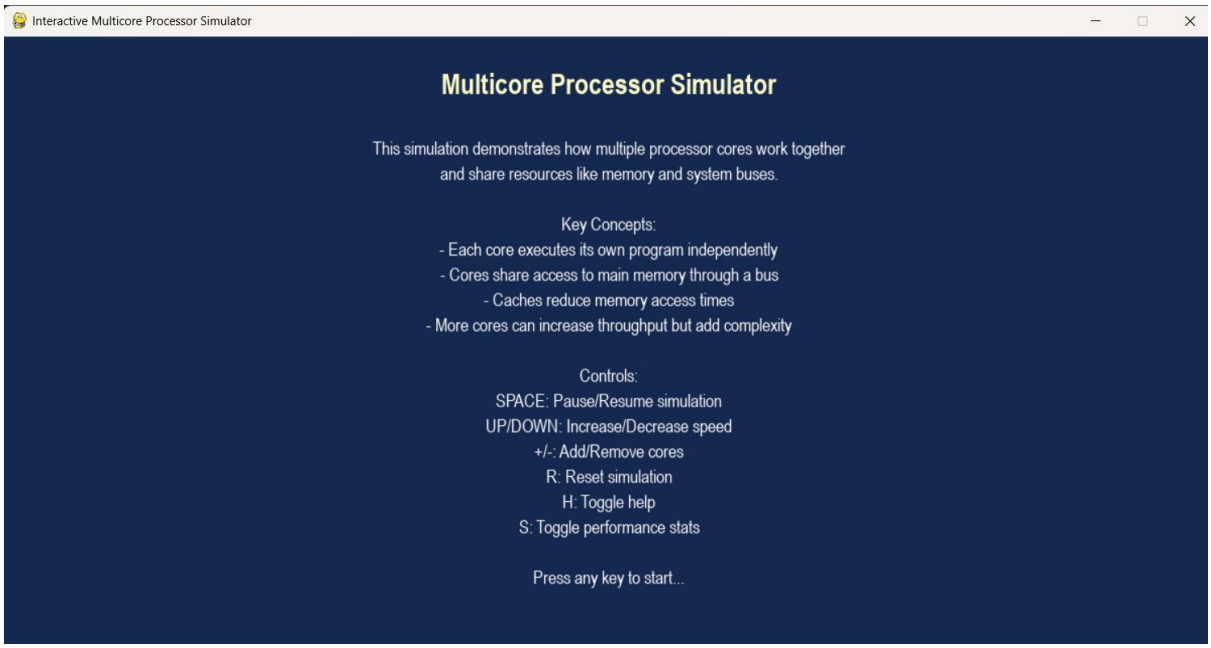

Figure 12: Simulation Instructions

The simulation has a variable number of cores, each of which runs independently an randomly selected program of instructions. The instructions are memory operations (LOAD, STORE) and arithmetic or logical operations (ADD, SUB, AND, OR, MOV). Each core has its own local registers, program counter, and cache but shares access to a common global memory space via a shared bus.

Some major features shown in the simulation are:

1. Core Activity Visualization: Every core is represented graphically using pulse animations and color coding. Its current instruction and execution status are displayed in real time.

2. Memory and Cache Behavior: The cores execute memory instructions that mimic cache hits and misses. The corresponding shared memory blocks are updated and displayed at the bottom of the screen. System Bus Arbitration: Access to memory is only granted if the bus is free, simulating real contention and delays among cores.

3. Performance Monitoring: The system monitors execution time and core usage. The program has a statistics mode which graphs speedup vs. time and utilization vs. time and provides a sense of multicore performance.

4. Interactive Controls: Pause/resume simulation, adjust simulation speed, change the number of cores, and view switching via keyboard shortcuts are all possible for the user.

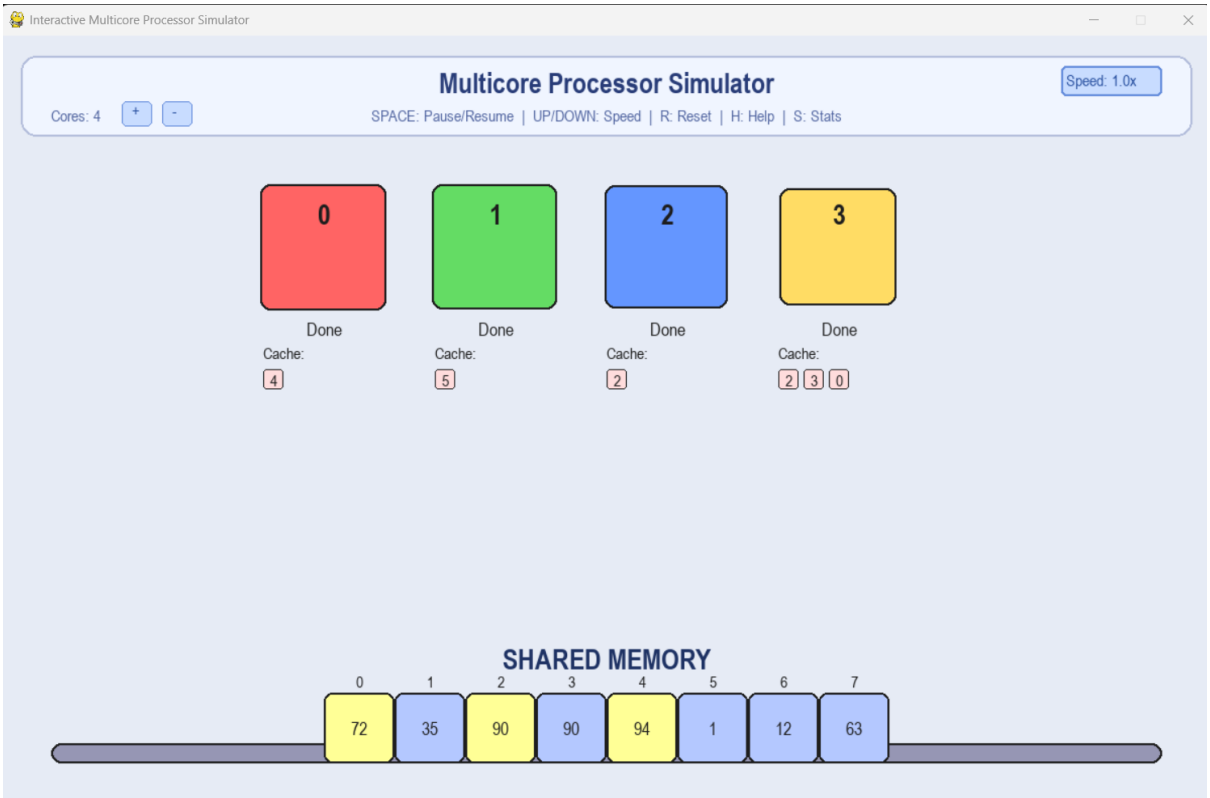

Figure 13: Simulation Interface

This simulation serves both as a learning tool and a visualization aid to understand key concepts in multicore processing, such as parallelism, resource sharing, and performance scaling. It reinforces the theoretical knowledge and hardware-based design by offering a software-based, interactive perspective on multicore systems.

4. Conclusion

This project was able to successfully illustrate the design and simulation of the multicore processor system in Logisim. Beginning with the development of the individual blocks like the logical unit, the arithmetic unit, and the register file, a complete CPU core was able to be built. The modularity of the parts enabled integrating the components into a scalable system such that it was possible to move from a single-core processor to a multicore system.

Critical elements of processor architecture, such as instruction fetch, decode, execution, and memory access, were realized and graphically represented. Particular care was taken in synchronizing shared resources, such as data memory, where a simple clock-based arbitration protocol provided safe and ordered access between cores.

Also, an interactive and dynamic visualization of multicore execution, memory access, and performance metrics was given by a Python-based software simulation. This complemented the theory and hardware simulation by demonstrating real-time parallel core behavior.

In conclusion, this project provided us with excellent exposure to the operation and challenge of multicore processors. It lays the groundwork for future improvements such as pipelining, more advanced memory controllers, and additional core support. Completing this project opened our eyes to the complexities of modern processor design, bridging the gap between theory and implementation.

4. References

OpenAI. (2025). ChatGPT Conversations on SE-ResNet. Retrieved from https://chatgpt.com/

https://miro.medium.com/v2/resize:fit:1200/0*5dVxHUhXoshujAhL

https://dashboard.snapcraft.io/site_media/appmedia/2023/03/logisim-icon-512.png

5. List of Figures